

Production Deployment Using AWS Elastic Beanstalk with Secure RDS Integration

Overview

This project demonstrates deploying a Python Flask Task Manager application on AWS using Elastic Beanstalk, with a secure MySQL RDS database integrated inside a custom VPC. Database credentials are stored securely using AWS Secrets Manager.

Table of Contents

1. [Application Development](#)
 2. [AWS Network Setup](#)
 3. [Security Groups](#)
 4. [RDS Database](#)
 5. [Secrets Manager](#)
 6. [IAM Role](#)
 7. [Deploy with Elastic Beanstalk](#)
 8. [Access Application](#)
 9. [Outcome](#)
-

Architecture

- Application runs inside Elastic Beanstalk (EC2)
 - RDS MySQL deployed within the same VPC
 - Database access restricted via security groups
 - EC2 instance connects securely to RDS
 - Credentials stored securely in AWS Secrets Manager
-

Step 1: Application Development (Local Setup)

The web application was developed and tested locally before deploying to the cloud.

Technologies Used: Python, Flask, MySQL, HTML

Project Folder Structure:

```
task-manager-app/  
  application.py  
  requirements.txt  
  procfile  
  templates/  
    index.html
```

Commands for Local Run:

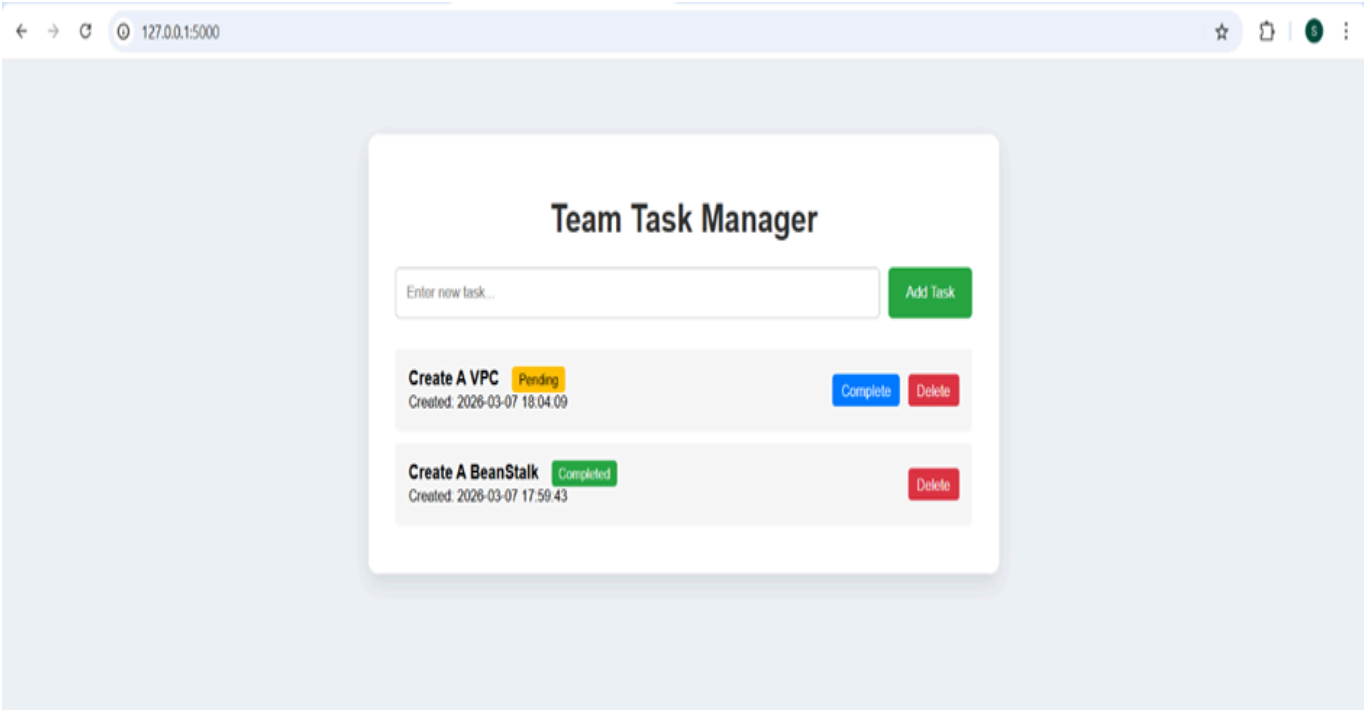
```
# Install dependencies
pip install flask pymysql

# Run the application
python application.py

# Open in browser
http://127.0.0.1:5000
```

The application was fully tested locally before any cloud deployment.

Screenshot - Local Application Running:



Step 2: AWS Network Setup (VPC Architecture)

A custom network architecture was created using VPC.

VPC Created: tech-vpc — CIDR: 10.0.0.0/16

Subnets Created:

Subnet Name	Type	CIDR Block
Web Subnet	Public	10.0.1.0/24
App Subnet	Private	10.0.2.0/24
DB Subnet 1	Private	10.0.3.0/24
DB Subnet 2	Private	10.0.4.0/24

Internet Gateway: Created and attached to `tech-vpc`

Route Table: Public route table created and associated with the Web Subnet.

Step 3: Create Security Groups

Navigate to AWS EC2 > Security Groups.

1. Application Security Group: `task-app-sg`

Inbound Rules:

Type	Port	Source
HTTP	80	0.0.0.0/0
SSH	22	My IP

Purpose: Allow users to access the application.

Screenshot - App Security Group:

Type	Port	Source
HTTP	80	0.0.0.0/0
SSH	22	My IP

2. Database Security Group: `task-db-sg`

Inbound Rules:

Type	Port	Source
MySQL	3306	task-app-sg

Purpose: Allow only the application server to access the database.

Screenshot - DB Security Group:

Type	Port	Source
MySQL	3306	task-app-sg

Step 4: Create RDS Database

Database Configuration:

Setting	Value
Engine	MySQL
Database Name	taskdb
Instance Class	Free Tier
VPC	tech-vpc
Subnet Group	db-subnet-group
Security Group	task-db-sg
Public Access	No

DB Subnet Group Created:

Step 5: Store Credentials in Secrets Manager

Database credentials were stored securely in AWS Secrets Manager instead of hardcoding them in the application.

Steps:

1. Go to AWS Secrets Manager
 2. Choose "Store a new secret"
 3. Select "Credentials for RDS database"
 4. Enter the database username and password
 5. Select the RDS instance
 6. Name the secret (e.g., `prod/taskdb/credentials`)
 7. Save
-

Step 6: Create IAM Role for Elastic Beanstalk

1. Go to IAM > Roles > Create Role
2. Select EC2 as the trusted entity
3. Attach the following policy: `SecretsManagerReadWrite`
4. Name the role: `aws-elasticbeanstalk-ec2-role`

Purpose: Allows the application running on EC2 to retrieve database credentials from Secrets Manager securely.

Step 7: Deploy Application Using Elastic Beanstalk

1. Create application: `task-manager-app`
2. Create environment: `task-manager-env`
3. Platform: Python
4. Upload the application package: `task-manager-app.zip`
5. Configure networking:
 - VPC: `tech-vpc`
 - Subnet: Web Subnet (public)
 - Security Group: `task-app-sg`
6. Assign IAM instance profile: `aws-elasticbeanstalk-ec2-role`

```

2026/03/08 21:23:40.530590 [INFO] Starting...
2026/03/08 21:23:40.530647 [INFO] Starting EBPlatform-PlatformEngine
2026/03/08 21:23:40.530667 [INFO] reading event message file
2026/03/08 21:23:40.531013 [INFO] Engine received EB command cfn-hup-exec

2026/03/08 21:23:40.590966 [INFO] Running command: /opt/aws/bin/cfn-get-metadata -s arn:aws:cloudformation:ap-south-1:964177142854:stack/awseb-e-mjgtmecpq-stack/3fb3aba0-1a63-11f1-88f4-0ac6b2b51ce1 -r
AWSEBAutoScalingGroup --region ap-south-1
2026/03/08 21:23:40.868234 [INFO] Running command: /opt/aws/bin/cfn-get-metadata -s arn:aws:cloudformation:ap-south-1:964177142854:stack/awseb-e-mjgtmecpq-stack/3fb3aba0-1a63-11f1-88f4-0ac6b2b51ce1 -r
AWSBeanstalkMetadata --region ap-south-1
2026/03/08 21:23:41.134038 [INFO] checking whether command bundle-log is applicable to this instance...
2026/03/08 21:23:41.134052 [INFO] this command is applicable to the instance, thus instance should execute command
2026/03/08 21:23:41.134055 [INFO] Engine command: (bundle-log)

2026/03/08 21:23:41.134103 [INFO] Executing instruction: GetBundleLogs
2026/03/08 21:23:41.134106 [INFO] Bundle Logs...
2026/03/08 21:23:41.269641 [INFO] Executing cleanup logic
2026/03/08 21:23:41.269805 [INFO] CommandService Response: ("status":"SUCCESS","api_version":"1.0","results":[{"status":"SUCCESS","msg":"Engine execution has succeeded.","returncode":0,"events":
[{"msg":"Instance deployment completed successfully.", "timestamp":1773005021269,"severity":"INFO"}]})

2026/03/08 21:23:41.269831 [INFO] Platform Engine finished execution on command: bundle-log

2026/03/08 21:24:15.084926 [INFO] Starting...
2026/03/08 21:24:15.084971 [INFO] Starting EBPlatform-PlatformEngine
2026/03/08 21:24:15.084991 [INFO] reading event message file
2026/03/08 21:24:15.085339 [INFO] Engine received EB command cfn-hup-exec

2026/03/08 21:24:15.144055 [INFO] Running command: /opt/aws/bin/cfn-get-metadata -s arn:aws:cloudformation:ap-south-1:964177142854:stack/awseb-e-mjgtmecpq-stack/3fb3aba0-1a63-11f1-88f4-0ac6b2b51ce1 -r
AWSEBAutoScalingGroup --region ap-south-1
2026/03/08 21:24:15.412280 [INFO] Running command: /opt/aws/bin/cfn-get-metadata -s arn:aws:cloudformation:ap-south-1:964177142854:stack/awseb-e-mjgtmecpq-stack/3fb3aba0-1a63-11f1-88f4-0ac6b2b51ce1 -r
AWSBeanstalkMetadata --region ap-south-1
2026/03/08 21:24:15.673415 [INFO] checking whether command tail-log is applicable to this instance...
2026/03/08 21:24:15.673431 [INFO] this command is applicable to the instance, thus instance should execute command
2026/03/08 21:24:15.673437 [INFO] Engine command: (tail-log)

2026/03/08 21:24:15.673480 [INFO] Executing instruction: GetTailLogs
2026/03/08 21:24:15.673483 [INFO] Tail Logs...
2026/03/08 21:24:15.673804 [INFO] Running command: tail -n 100 /var/log/nginx/access.log
2026/03/08 21:24:15.675878 [INFO] Running command: tail -n 100 /var/log/nginx/error.log
2026/03/08 21:24:15.677626 [INFO] Running command: tail -n 100 /var/log/web.stdout.log
2026/03/08 21:24:15.679227 [INFO] Running command: tail -n 100 /var/log/eb-engine.log

.....
/var/log/eb-hooks.log
.....

```

Screenshot - Beanstalk Configuration:

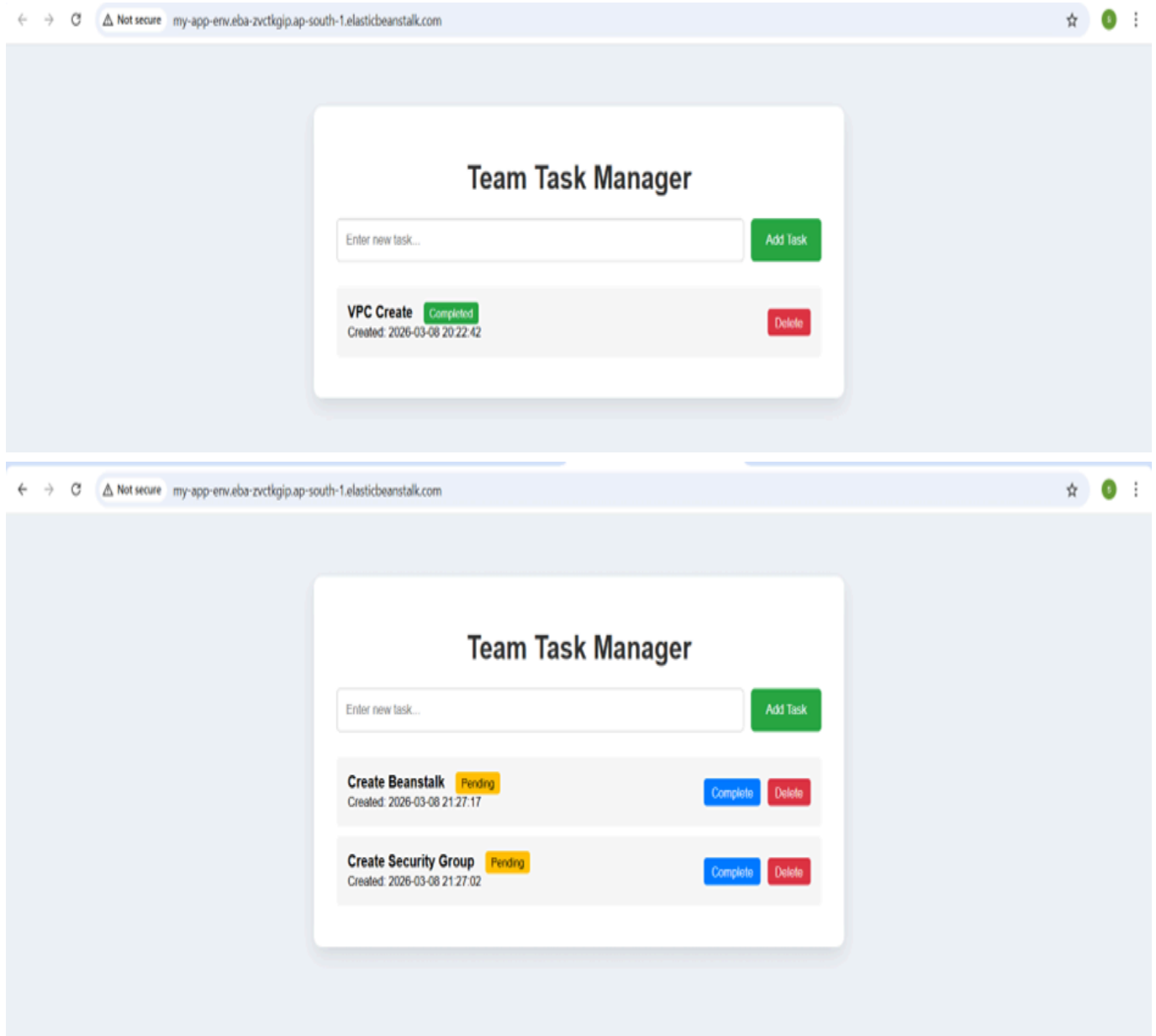
Step 8: Access Application

Elastic Beanstalk automatically generates a domain URL after deployment.

Application URL:

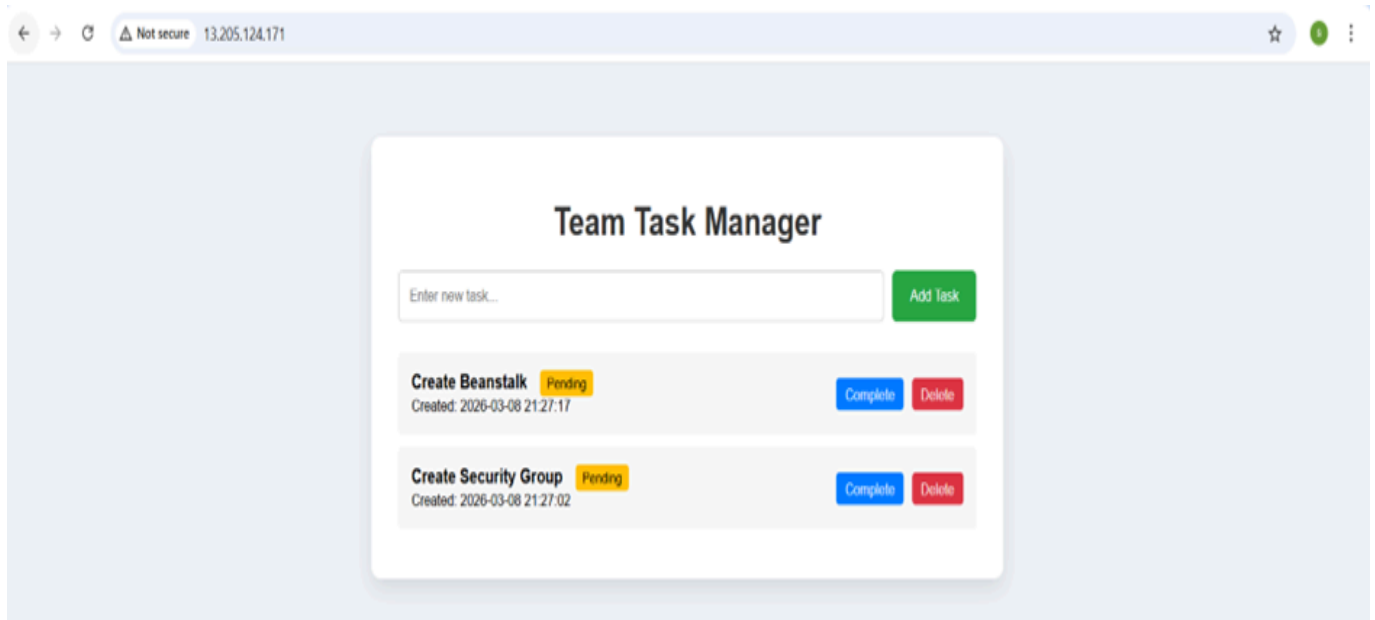
```
http://task-manager-env.eba-zvctkgip.ap-south-1.elasticbeanstalk.com/
```

Screenshot - Application Live:



An EC2 instance is automatically created by Elastic Beanstalk. The application can also be accessed via the EC2 instance's public IP.

Screenshot - Access via Public IP:



Outcome

The Python Task Manager application was successfully deployed on AWS with:

- Secure networking using a custom VPC with public and private subnets
 - Managed deployment using Elastic Beanstalk
 - Secure database access restricted by security groups
 - Credentials managed securely using AWS Secrets Manager
 - Scalable and production-ready architecture using AWS services
-